

SAFESPACE AI

ENTERPRISE SYSTEM ARCHITECTURE & INTEGRATION SPECIFICATION

Real-time Anti-Cyberbullying & Message Safety Middleware

VERSION: 1.0.0 (Production-Ready)

DATE: August 18, 2026

PRIMARY AI MODEL: Google Gemini 3.6 Flash (v1beta Direct API)

DATABASE STACK: Supabase PostgreSQL with Real-time Replication

PUBLISHER: SafeSpace AI Engineering Division

1. Executive Summary & Problem Space

Online platforms experience rising rates of cyberbullying, hate speech, and harassment. This behavior degrades community engagement and creates legal liabilities for operators. Conventional content moderation systems rely on post-hoc reporting or crude keyword filters that are easily bypassed by phonetic spelling, leetspeak, or regional slurs.

SafeSpace solves this problem by inserting a real-time safety evaluation layer directly into message ingestion channels. SafeSpace acts as middleware, intercepting chats from Instagram, WhatsApp, X/Twitter, and YouTube Live Stream chats. By routing comments to our safety classification engine before delivery, SafeSpace catches harassment, classifies severity index levels, and flags violations instantly.

The SafeSpace Solution Paradigm:

- Pre-Delivery Interception: Messages are filtered prior to user delivery, protecting community health.
- Context-Aware AI: Leverages Gemini 3.6 Flash models capable of understanding language nuances and slurs.
- Multilingual Support: Scans Indian code-mixed languages (like Hinglish or Kanglish) in Latin transcripts.
- Frictionless Integration: Exposes webhooks, allowing retrofitting to legacy architectures in under 10 lines of code.

2. Core System Modules & Capabilities

The SafeSpace platform is constructed from four main modules, each addressing a distinct phase of the moderation lifecycle: Ingestion, Classification, Analytics, and Governance.

A. Ingestion Hub & Sandbox Simulator

The ingestion hub registers social channel connectors, generates cryptographic signing secrets, and tracks call frequency. The visual Sandbox simulator lets team members test direct messaging feeds and watch the interception pipeline in real time.

B. AI Safety Classifier

Directly queries Google Gemini 3.6 Flash models using structured JSON schemas. It maps threats, harassment, hate speech, and sexual content to severity scales, yielding natural language explanations.

C. Real-time Incident Dashboard

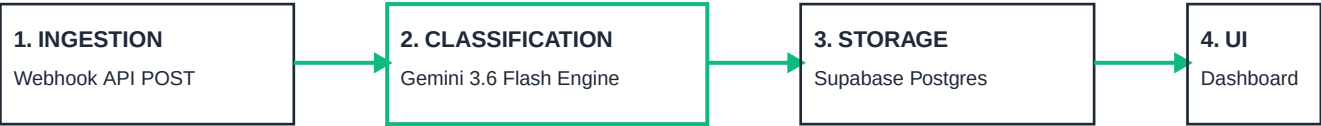
Leverages Supabase Postgres replication channels. Whenever a message is flagged as high-risk, it creates an incident file. This incident appears on moderator feeds instantly without requiring page refreshes.

D. Governance & Audit Trail

Logs all moderation events, role updates, and override logs. This keeps moderator teams accountable.

3. High-Level System Architecture

The diagram below displays the SafeSpace middleware components and data synchronization loops:



**Data
Flow Ex
ecution
Stages:**

- 1. A social application forwards an incoming message payload to /api/webhook.
- 2. The SafeSpace engine checks source authorization keys, verifies structural integrity, and invokes the AI engine.
- 3. Gemini 3.6 Flash analyzes the content and replies with classification details (labels, severity, explanation).
- 4. Logs are written to Supabase tables. If flagged, content items and incident records are generated.
- 5. Supabase broadcasts the new records to authenticated clients, populating moderation dashboards instantly.

4. Data Ingestion Pipeline & Hook Routing

The SafeSpace data ingestion pipeline is built to handle high-frequency incoming message streams. Below is the block layout of the ingestion phase before classification:

A. Security & Signature Verification

Every registered Ingestion Source carries a generated cryptographic signing secret. When external webhooks hit SafeSpace, their request headers include a signature token to authenticate the payload. This prevents spoofing and API abuse.

B. Message Sanitization

Prior to model query formatting, raw text payloads undergo character cleaning. This strips non-printable unicode anomalies and malicious escape attempts while retaining emojis and punctuation critical for tone classification.

C. Transactional Tracking

Each event update is logged inside the database. SafeSpace updates ingestion source metadata, tracking message counters and last-event timestamps to facilitate API analytics and rate limiting.

5. AI Safety Classifier Mechanics

The classification module maps text to a structured safety schema. SafeSpace instructs Gemini to bypass verbose textual replies and output data strictly adhering to a parsed JSON format.

Core Classification Prompts & Rules:

1. Safety Evaluation: Evaluate the input message for cyberbullying, threats, sexual harassment, and general slurs.
2. Category Mapping: Tag matching labels: harassment, threat, hate_speech, insult, sexual, or spam.
3. Severity Scoring: Assign a severity index (safe, low, medium, high) based on target directness and language intensity.
4. Action Advice: Output moderation advice (allow, flag, warning, hide, block).
5. Explainable AI: Detail the reasoning in the language of evaluation.

Output JSON Contract:

```
{
  "labels": ["harassment", "insult"],
  "severity": "high",
  "confidence": 0.98,
  "explanation": "Targeted insult using regional slurs.",
  "recommended_action": "flag_user",
  "requires_review": true
}
```

6. Code-Mixed & Multilingual Context Mappings

Online harassment in regional communities often uses code-mixed text (e.g., Hinglish, Kanglish) written in Latin characters. Conventional classifiers fail here because these words do not exist in standard English vocabularies.

A. Latin Script Slur Mapping

SafeSpace safety prompts feed Google Gemini 3.6 Flash with regional slur mappings. This permits the model to recognize phonetic spellings of Hindi or Kannada insults (e.g. "chutiya", "saala", "bolimaga") and flag them based on severity.

B. Context-Dependent Evaluation

The engine evaluates the full context of sentences rather than relying on blocklists. If a word is used in friendly banter, it is rated safe. If it targets an individual to cause harm, the severity level goes up.

C. Explanation Localization

When the classifier flags a message, the generated explanation translates the context of the slur into English. This helps moderators who do not speak that dialect make accurate decisions.

7. Database Schema Reference - Ingestion & Posts

SafeSpace tables are hosted on Supabase Cloud. Below are the schema definitions for the ingestion layer:

Table: public.ingest_sources

Stores registered social applications connecting to the SafeSpace pipeline:

- id (uuid, PRIMARY KEY): Unique identifier for the application source.
- created_at (timestamp): Time when connection was registered.
- name (text): Application display name (e.g., "YouTube Live").
- platform (text): Target messaging channel (instagram, whatsapp, youtube, twitter, custom).
- signing_secret (text): Cryptographic key used to verify webhook calls.
- event_count (integer): Count of events processed through this source.
- active (boolean): State toggle to allow or pause ingestion.

Table: public.ingested_posts

Main log repository for incoming message payloads:

- id (bigint, PRIMARY KEY): Auto-incrementing identifier.
- source_id (uuid): Foreign key linking back to public.ingest_sources.
- platform (text): Redundant platform label for fast dashboard filtering.
- external_id (text): Message ID mapped from the external social network.
- author_handle (text): Sender username.
- body (text): Processed clean text body.
- severity (app_severity enum: safe, low, medium, high).
- confidence (numeric): Classification confidence scale.
- requires_review (boolean): Toggled true if flagged by AI models.

8. Database Schema Reference - Predictions & Actions

Below are the schema specifications for content review and incident tracking:

Table: public.content_items

Stores flagged messages requiring human moderator review:

- id (bigint, PRIMARY KEY): Unique identifier.
- created_at (timestamp): Timestamp of incident logging.
- body (text): Highlighted text body with metadata.
- severity (app_severity): High-level severity classification.
- visibility_status (status enum: visible, hidden, flagged_only).
- requires_review (boolean): Review flag status (toggled false upon review).

Table: public.model_predictions

Stores model prediction metadata details:

- id (bigint, PRIMARY KEY): Unique identifier.
- content_id (bigint): Link to target public.content_items.
- model_version (text): Version of classification model (e.g. gemini-3.6-flash).
- labels (text[]): Applied classification categories (threat, hate_speech, etc).
- confidence (numeric): Confidence percentage.
- explanation (text): Natural language reasoning explanation.
- recommended_action (text): Action recommendations (allow, flag_user, delete).

Table: public.notifications

System alerts dispatching real-time notifications to staff dashboard layouts:

- id (bigint, PRIMARY KEY): Unique identifier.
- audience (text): Target audience role (staff, admin).
- kind (text): Notification category (new_flag, system_alert).
- title (text): Notification subject line.

9. Row Level Security & User Access Rules

To maintain high safety and secure operational parameters, SafeSpace leverages PostgreSQL Row Level Security (RLS) policies.

A. Ingestion Source Access Rules

The `ingest_sources` table holds connection metadata and cryptographic secrets. Read and write permissions are granted to authenticated accounts and anonymous roles (enabling webhook registrations) while hiding master fields from normal public users.

B. Message Logging Access Rules

The `ingested_posts` table allows authenticated webhooks to write records. Reading logged chats is allowed for staff members and sandbox simulator panels, keeping messages private.

C. Moderator Action Access Rules

Moderator operations on `audit_logs` and `content_items` require specific staff roles (moderator, auditor, admin). Regular users are blocked from viewing audit logs or overrides.

10. REST Webhook API Specification

SafeSpace exposes a single REST webhook endpoint callback allowing external applications to push messages in real-time.

HTTP Request Contract:

- Endpoint Path: /api/webhook
- HTTP Method: POST
- Content-Type: application/json

Request Body Format (JSON):

```
{
  "sourceId": "8f87e5b3-3a72-4d2a-8742-5e921d283aa0",
  "authorHandle": "chat_user_99",
  "targetHandle": "celebrity_profile",
  "body": "Hey, you look so ugly and fat. Go delete your account."
}
```

Response Body Format (JSON):

```
{
  "ok": true,
  "post": {
    "id": 84,
    "source_id": "8f87e5b3-3a72-4d2a-8742-5e921d283aa0",
    "platform": "instagram",
    "author_handle": "chat_user_99",
    "body": "Hey, you look so ugly and fat. Go delete your account.",
    "severity": "high",
    "requires_review": true
  }
}
```

11. Meta Platforms Integration Guidelines

Connecting production feeds from Instagram and WhatsApp requires configuring webhooks in the Meta Developer Portal.

A. Instagram Direct Messages Webhook

To intercept Instagram direct messages, create a Meta Business Developer application, select the "Instagram Graph API" product, and associate your app with an Instagram Professional account. Set the webhook callback URL to point to SafeSpace's /api/webhook path. Subscribe to the "messages" event topic. The Meta API will dispatch message payloads as they land, and SafeSpace will evaluate them instantly.

B. WhatsApp Business Webhook Setup

To monitor WhatsApp Business numbers, configure the "WhatsApp" product in your Meta developer app dashboard. Set the webhook target to the SafeSpace endpoint. In the webhook subscription page, check the "messages" topic checkbox. SafeSpace will process text payloads from customers and store flagged harassment violations for customer support audits.

12. Streaming & Live Chat API Connectors

Live streaming channels (YouTube Live and X/Twitter DMs) require background polling workers to capture chat message events.

A. YouTube Live Stream Chat Polling

YouTube's Data API v3 does not push message webhooks directly. Instead, a lightweight listener background script queries the `liveChatMessages/list` endpoint using the active `LiveChatId`. This polling script retrieves comment logs every 2 seconds, formats payloads, and redirects comments to the SafeSpace API.

B. X / Twitter Account Activity API

X uses the Account Activity API (AAA) to dispatch account activity events. Developers register the SafeSpace webhook URL inside the Twitter developer control panel. Twitter sends direct message (DM) payload alerts to SafeSpace in real-time, enabling immediate safety evaluation.

13. Compliance, Privacy, & Data Sanitization

Handling user communication feeds requires strict compliance with international privacy rules (GDPR, CCPA) and regional data protection laws.

A. Data Anonymization

SafeSpace protects user identities by storing handle representations instead of real-world names. Database tables do not contain phone numbers, email addresses, or physical location keys.

B. Automated PII Stripping

The sanitization middleware scans message bodies for Personally Identifiable Information (PII) like credit card numbers, social security keys, or passwords. These elements are redacted prior to model classification.

C. Audit Retention Limits

SafeSpace implements configurable data retention policies. Safe messages are purged from logs after 30 days, while flagged incidents are archived for moderator review and deleted once resolved.

14. Product Roadmap & Future Scope

SafeSpace plans to build on its core moderation pipeline with the following upcoming features:

A. Multimodal Content Classification

Future updates will expand the pipeline to evaluate images, audio clips, and short video attachments sent inside messaging feeds. The system will leverage Gemini's multimodal inputs to catch visual harassment.

B. Browser Extension Integration

We are developing a browser extension that integrates SafeSpace directly into client-side social feeds (Instagram Web, WhatsApp Web). This extension will blur abusive content in the browser, protecting users without requiring server modifications from social platforms.

C. Continuous Model Tuning Loops

We will build a feedback loop that uses resolved moderation incidents to tune our models. This will continuously improve classification accuracy and reduce false positives over time.

